

Memory Management in TinyML

Static Memory Allocation Strategy:

- No `malloc()/free()` to avoid heap fragmentation
- Pre-planned memory layout for long-running applications
- Fixed-size arena passed at initialization
- Immediate error if arena too small

Memory Arena Concept:

```
// Application provides memory arena
constexpr int kTensorArenaSize = 60 * 1024;
uint8_t tensor_arena[kTensorArenaSize];

// Framework uses arena for all allocations
tflite::MicroInterpreter interpreter(model, resolver,
                                     tensor_arena, kTensorArenaSize);
```

uint8_t is a typedef in C and C++ that represents an unsigned 8-bit integer

Benefit: Predictable memory usage, no runtime allocation failures

Forward Mode

Applications in TinyML

- **Online Learning:** Single example updates with predictable memory
- **Real-time Sensitivity:** Track sensor importance in real-time
- **Edge Deployment:** Constant resource usage enables deployment
- **Example:** Sensor fusion with importance weighting

```
def analyze_sensor_importance(model, sensor_data):  
    # Track how each sensor affects the prediction  
    # Forward mode: predictable memory usage  
    for sensor_i in range(len(sensor_data)):  
        sensitivity = compute_forward_derivative(model,  
        sensor_data, sensor_i)  
        importance_scores[sensor_i] = sensitivity
```

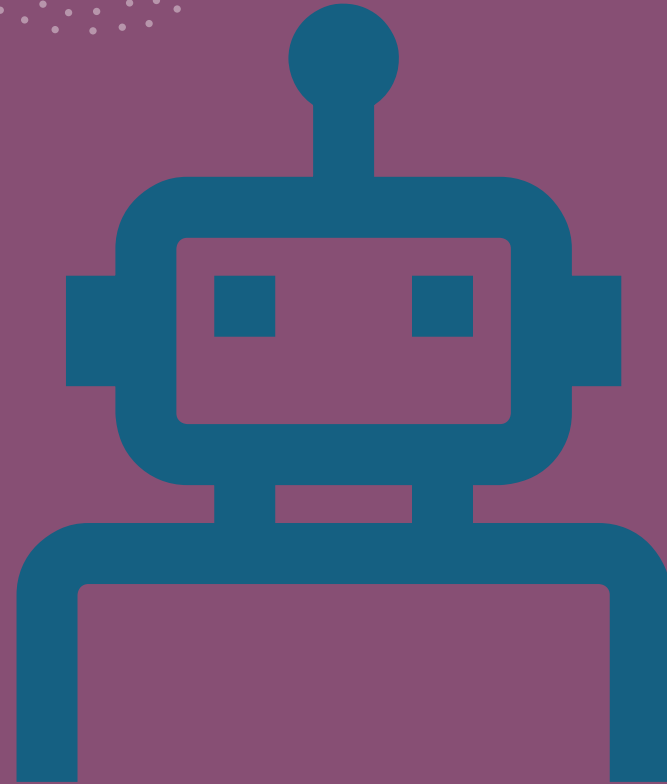
Gradient Checkpointing

- **Strategy:** Store fewer intermediates, recompute during backward
- **Trade-off:** $\sqrt{n}$ memory reduction for $\sim 33\%$ computation increase
- **Example:** Store every k -th layer, recompute the rest
- **Implementation:** Framework manages storage/recomputation automatically
- **Result:** Enables training of much larger models



TinyML Gradient Computation

- • **TinyML Characteristics:**
- Energy: $< 1 \text{ mW}$
- Memory: Kilobytes total
- No dynamic allocation
- Real-time requirements
 - **Challenge:** Wake word detection with federated learning
 - **Solution:** Aggressive checkpointing with full recomputation



Memory-Efficient AD Implementation

- **Micro-batching:** Process one example at a time
- **Quantized Gradients:** 16-bit or 8-bit gradient computation
- **Static Allocation:** Pre-allocate all memory at compile time
- **Template Optimization:** Generate specialized code for specific architectures
- **Result:** Complete AD system in <5KB binary

Federated Learning Case Study

Scenario: On-device wake word detection

Requirements:

- Local gradient computation on audio
- <32KB total RAM
- Real-time operation
- Federated participation
 - **Implementation:** Forward-only checkpointing + quantization
 - **Results:** 5-10x computation, fits constraints



RAM Usage Analysis

```
// Static allocation approach
uint8_t tensor_arena[50 * 1024]; // 50KB arena

tflite::MicroInterpreter interpreter(
    model, resolver, tensor_arena,
    sizeof(tensor_arena), error_reporter);

// Check if arena is sufficient
if (interpreter.AllocateTensors() != kTfLiteOk) {
    // Arena too small - increase size
}
```

Dynamic Memory Requirements

TensorFlow Lite Memory Arena:

The arena holds temporary activation tensors during inference

Arena size calculation:

- `arena_size = max_simultaneous_tensor_memory`

Typical requirements by model type:

- **Simple_dense_model**: 5-20KB
- **Small_CNN**: 20-100KB
- **Efficient_transformer**: 50-200KB

Application Memory:

- Input/output buffers
- Preprocessing workspace
- Communication buffers
- Business logic variables